

Документация HandCheck

Индекс документов проекта. Требования ТЗ: «ссылка на документацию проекта в формате doc/pdf».

Обязательное по ТЗ

Требование ТЗ	Документ
Функциональная и компонентная архитектура	ARCHITECTURE.md — разделы «Функциональная архитектура» и «Компонентная архитектура»
Механика тестирования и обоснование устойчивости	TESTING.md
Механика подбора и логика категоризации	MATCHING.md
Собственная процедура валидации и результаты	VALIDATION.md + context/metrics-assessment.json
Схема интеграции с реестром ФСП	FSP-INTEGRATION.md
Описание API	API.md + openapi.yaml
Использованные библиотеки и их версии	DEPLOYMENT.md , раздел «Библиотеки и версии»
Пошаговая инструкция сборки, развёртывания и локального запуска	DEPLOYMENT.md

Прочее

Документ	О чём
TZ.md	сжатое ТЗ ФСП

Документ	О чём
presentation/T3-ФСП-2026.txt	полный текст ТЗ из PDF ФСП 2026
FUNCTIONAL-COVERAGE.md	покрытие ТЗ по пунктам: что сделано, чем подтверждено, чего нет
PRESENTATION.md	текст презентации и ответы на вопросы жюри
presentation/HandCheck-ФСП-2026.pptx	презентация в официальном шаблоне ФСП
presentation/HandCheck-ФСП-2026.pdf	та же презентация в PDF
DATA-MODEL.md	таблицы и приватность
UX.md	маршруты и тексты интерфейса
MCP.md	MCP-интеграция для ИИ-клиентов
JURY-DEMO.md	клик-маршрут демонстрации жюри
IMPLEMENTATION-PLAN-rev5.md	каноническая спецификация rev.5
../README.md	быстрый старт и ссылки
../context/audits/	отчёты функционального аудита

Как поддерживать документацию

```
python3 scripts/build-documentation-pdf.py      # все docs/*.md → docs/Documentation-HandCheck.pdf
node scripts/functional-audit.js                # отчёты в context/audits/
METRICS_JSON=context/metrics-assessment.json node scripts/assessment-metrics.js
```

Сводный PDF собирается из 13 разделов в порядке: индекс → покрытие ТЗ → ТЗ → архитектура → тестирование → подбор → валидация → интеграция ФСП → API → модель данных → деплой → MCP → демо.

Ссылки для жюри (в репозитории)

Что	Файл в репозитории
Индекс документации	docs/DOCUMENTATION.md
Сводная документация одним файлом	docs/Documentation-HandCheck.pdf
Презентация (PDF / PPTX)	docs/presentation/HandCheck-ФСП-2026.pdf · pptx
Короткая презентация (13 слайдов)	docs/presentation/HandCheck-overview.pdf
Полный текст ТЗ ФСП	docs/presentation/ТЗ-ФСП-2026.txt
Скриншоты интерфейса	handcheck-ui/presentation/

Ссылки для жюри

Что	Ссылка
Развёрнутый прототип	https://handcheck.baski.pro
Репозиторий	https://github.com/BaskovKonstantin/handcheck
Презентация	docs/presentation/HandCheck-ФСП-2026.pptx / .pdf
Документация	этот раздел и файлы выше
Демо-вход	cafe@demo.local (работодатель), anna@demo.local , boris@demo.local (кандидаты), код подтверждения 000000

Покрытие ТЗ функциональностью

Документ отвечает на вопрос «весь ли функционал ТЗ реализован» и на что опирается каждое утверждение. Источники требований: [docs/TZ.md](#) (сжатое ТЗ) и полный текст PDF ФСП-2026 — [docs/presentation/ТЗ-ФСП-2026.txt](#).

Как проверялось

Два независимых прогона [scripts/functional-audit.js](#):

```
# полный (пишущий) прогон на чистой БД
PORT=8899 DB_PATH=/tmp/hc-audit.sqlite DEMO_MODE=1 npm start &
BASE_URL=http://127.0.0.1:8899 node scripts/functional-audit.js

# читающий прогон по прод у (ничего не пишет в jury-БД)
BASE_URL=https://handcheck.baski.pro AUDIT_MODE=readonly node scripts/functional-audit.js
```

Плюс алгоритмические проверки качества теста и подбора:

```
npm run validate # validate:assessment + validate:matching + validate:privacy
npm test # 347 pass / 9 fail – те же 9 падений есть на чистом HEAD
```

Отчёты: [context/audits/2026-10-09-full-local.md](#) (41 PASS, 0 FAIL) и [context/audits/2026-10-09-prod-readonly.md](#) (8 PASS, 0 FAIL, прод на коммите `f90eb4b`).

Сводка

Область ТЗ	Статус	Чем подтверждено
Механика подбора (35%)	Реализована и проверена	E-04...E-13, C-17...C-19
Тест, валидация, категории (25%)	Реализованы и проверены	C-08...C-14, <code>npm run validate:assessment</code>
UI (10%)	Два кабинета + демо-вход	R-08, скриншоты в <code>context/design-refs/</code>
Техника (7%)	Docker, OpenAPI, секреты вне git	README, <code>.github/workflows/</code> , X-05
Полнота и доки (6%)	README, ARCHITECTURE, VALIDATION, аудит	этот документ
Доп. функционал (7%)	Частично	X-01...X-09
Питч (10%)	См. презентацию	<code>docs/PRESENTATION.md</code>

Механика подбора — ядро жюри (35%)

Требование ТЗ	Статус	Проверка
Потребность → подборка по категории	✓	E-05: все строки <code>categoryLabel</code> = backend, у каждой <code>explanation</code>
Категория ставится только после теста	✓	E-03: у кандидата без подтверждённой категории подборка пустая
Ранжирование внутри категории	✓	E-06 (порядок стабилен), <code>validate:matching</code> M1/M2 (score + буст ФСП)
Фильтры не «съедают» подборку	✓	E-07: после <code>?stack=</code> и <code>?fsp=1</code> состав подборки не изменился
Поиск по банку: специализация, грейд, стек, ФСП	✓	E-08: <code>spec, grade, stack, fsp, q, status</code> ; фильтр только сужает
Приглашение с вилкой от-до, без обязательной вакансии	✓	E-09 (без вилки → 400 <code>salaryRange</code>), E-10 (201, дубль → 409)
Статусы приглашений	✓	E-11 (<code>sent</code>), C-18 (<code>accepted</code>), C-19 (<code>declined</code>) у обеих сторон
Контакты скрыты до ассепт	✓	C-17 (403), E-11 (в JSON приглашения нет <code>phone/email</code>), R-05 (403 на проде)
Контакты раскрываются после ассепт	✓	E-12: <code>/contacts</code> → 200 + <code>phone</code> в списке приглашений
Инициатива у работодателя	✓	Кандидат не может создать потребность/приглашение (C-07 → 403)

Дополнительно: колода (`deck/next`) с решениями «отложить/отказать», которые блокируют повторное приглашение (E-13), и короткий тест работодателя, назначаемый только после принятого приглашения (X-02).

Кандидат (MVP)

Требование ТЗ	Статус	Проверка
Регистрация + подтверждение email	✓	C-01 (регистрация → код → сессия), C-03 (неверный код → 400)
Согласие на обработку данных	✓	C-02 и C-08: без <code>privacyConsent</code> регистрация и старт батареи → 400
Профиль кандидата	✓	C-06: чтение и запись (имя, стек, телефон, контактный email)
Опрос → грейд → тест → категория	✓	C-09: <code>formKey</code> A/B, 9 шагов; C-12: батарея до конца
Статус категории и грейда	✓	C-12: <code>confirmed</code> или <code>unconfirmed</code> + <code>retakeAt</code> ; C-14: заявленный грейд в истории
Пересдача с лимитом частоты	✓	C-13: повторный старт → 409 <code>cooldown</code> , <code>retakeAt</code> = +30 дней (<code>GRADE_COOLDOWN_DAYS</code>)
Грейд не режется принудительно вниз	✓	Категория = заявленный грейд при прохождении <code>cutoff</code> ; при непрохождении — <code>unconfirmed</code> , а не понижение
Входящие приглашения: принять/отклонить	✓	C-18, C-19
Видит только свои данные	✓	C-07 (роль), C-15 (нет <code>integrity</code> , <code>trust_ok</code> , ответов в выдаче)
Связь с ФСП ID	✓	Поле <code>fsp_id</code> в схеме, буст <code>fsp_boost</code> в ранжировании, кейс «нет истории ФСП» работает — <code>validate:matching</code> МЗ

Работодатель (MVP)

Требование ТЗ	Статус	Проверка
Профиль компании	✓	E-01: пустое название → 400; без профиля приглашение не создаётся
Потребность с описанием	✓	E-02: валидация, дубль названия → 409, обновление
Подборка с обоснованием	✓	E-05
Поиск по банку	✓	E-08
Приглашение с оффером и ЗП от-до	✓	E-09, E-10
Статусы приглашений	✓	E-11, C-18, C-19
Контакты не видны до ассепт	✓	C-17, R-05

Тест и валидация (25%)

Требование ТЗ	Статус	Проверка
Свой устойчивый подход, не «одно на всех», не «уникальный LLM на каждого»	✓	Банк 9 батарей (3 специализации × 3 грейда) + параллельные формы A/B, дедлайны, черновики, телеметрия
Шаг за шагом, без пропусков	✓	C-10: <code>not_current_task</code>
Черновик и таймеры	✓	C-11: черновик возвращается после reload; <code>deadline_passed</code> в коде
Cutoff по грейду (0.55 / 0.68 / 0.78)	✓	<code>npm run validate:assessment</code> M1
Эквивалентность форм A/B	✓	<code>validate:assessment</code> M2
Устойчивость к утечкам (2 формы на ячейку)	✓	<code>validate:assessment</code> M3

Требование ТЗ	Статус	Проверка
Лимит пересдачи	✓	C-13
LLM — только вспомогательно (генерация черновиков)	✓	X-04: <code>POST /api/assessment/generate</code> + публикация

Техника, безопасность, приватность

Требование	Статус	Проверка
Контакты скрыты до ассерт на уровне API	✓	C-17, E-12, <code>validate:privacy</code> P1b
API-токен только для MCP, не для REST	✓	X-05: REST с токеном без сессии → 403
Аудит вызовов ИИ-клиента	✓	X-05/X-06: <code>whoami</code> в аудите, токен отзывается
Секреты вне git	✓	<code>.env</code> на сервере, в репозитории только <code>.env.example</code>
152-ФЗ	✓	C-16: публичное уведомление; согласие обязательно на регистрации и на старте батареи
Docker + OpenAPI	✓	<code>docker-compose.yml</code> , <code>openapi.yaml</code> , деплой по <code>main</code>

Что не реализовано (и это осознанно)

Пункт ТЗ	Статус	Комментарий
Вакансии + отклик кандидата	✗ не сделано	Каталог <code>app/modules/vacancies/</code> пуст, роутов нет. ТЗ относит к «желательно, не блокер». Механика приглашений работает без вакансий (E-10)
Регулярные короткие задания от работодателя	⚠ частично	Короткие тесты, назначаемые кандидату и проходимые после ассерт, работают (X-01...X-03). Периодической выдачи «по расписанию» нет
PDF-резюме → разбор	✗ не сделано	Зависимостей для PDF/OCR в проекте нет. ТЗ:

Пункт ТЗ	Статус	Комментарий
(OCR)		«желательно позже». Форма профиля заполняется вручную
Автогенерация PDF-профиля кандидата	✗ не сделано	Пункт обязательного списка кабинета кандидата; экспорт профиля в PDF не реализован
Реальная интеграция с реестром ФСП	⚠ концепция + заготовка	Схема адаптера, модель данных, ранжирование и режим деградации — в docs/FSP-INTEGRATION.md . Сетевого вызова нет: публичного API не существует. Кейс «нет истории ФСП» работает
ATS-интеграции	✗ вне кода	В ТЗ — «за рамками кода, нужна концепция в доке»
Антифрод вакансий	✗ вне кода	Нужна концепция; базовые ограничения уже есть: контакты после ассерт, запрет повторных приглашений, решения колоды учитываются
Адаптивная мобильная версия	✓	Интерфейс свёрстан под 390 px, проверено скриншотами в handcheck-ui/presentation/

Найденные и починенные дефекты

Дефект	Симптом	Исправление
Мёртвый эндпоинт генерации заданий	POST <code>/api/assessment/generate</code> отдавал 403 любой ролью: <code>tasks/router</code> монтировался после <code>assessment/router</code> , а тот гейтит всё под <code>requireRole("candidate")</code>	Роутер заданий смонтирован перед <code>assessment-роутером</code> в app/server.js
Устаревшая строка в документации	В <code>ARCHITECTURE.md</code> числился GET <code>/api/meta</code> , которого в коде нет	Строка удалена, эндпоинт не заявляется

Команды проверки

<code>npm test</code>	# тесты
-----------------------	---------

```
npm run validate # качество теста/подбора/приватности
node scripts/functional-audit.js # функциональный аудит (нужен запущенный
стенд)
BASE_URL=https://handcheck.baski.pro AUDIT_MODE=readonly node scripts/functional-audit.js
```

HandCheck — ТЗ (сжато для реализации)

Источник: ФСП спец. трек 2026, PDF 9 стр. Полный текст у оператора: `OpenBas/context/fsp-
spec-track-tz-extract.txt`.

Смысл продукта (нельзя ломать)

Платформа **не** витрина чужих вакансий и **не** «кандидат откликается вслепую».

1. Кандидат проходит **опрос + тест на заявленный грейд**.
2. Система присваивает **категорию = специализация × грейд**.
3. Работодатель ищет **категорию / банк**, сам шлёт **приглашение с вилкой ЗП**.
4. Контакты кандидата **скрыты**, пока он не принял приглашение (или сам не откликнулся — доп. сценарий).

Ориентир механики: LHH / Hired — инициатива у работодателя. Это **главный критерий жюри (35%)**.

MVP обязательно

Кандидат

- Профиль (форма; PDF+OCR — желательно позже)
- Опрос отрасли/специализации → выбор грейда → тест → категория
- Статус категории/грейда, история теста, пересдача выше/ниже с **лимитом частоты** (по умолчанию 30 дней)
- Входящие приглашения: принять / отклонить + статусы
- Приватность, согласие на данные, опциональная связь с **FSP ID**
- Видит только свои данные

Работодатель

- Профиль компании
- Описание потребности → подборка категорий/кандидатов **с обоснованием**

- Поиск по банку: специализация, грейд, категория, стек, наличие ФСП
- Приглашение с оффером и ЗП **от-до Р** без обязательной вакансии
- Статусы приглашений
- Контакты кандидата не видны до ассепт

Правила подбора

- Выдача по **категории теста**, не по самоописанному резюме
- Ранжирование внутри категории: результат теста + буст ФСП (если есть)
- Фильтры не должны «съедать» уже полученную подборку
- ЗП «от-до» обязательна в приглашении (и в вакансии, если вакансии есть)

Регистрация

- Email + подтверждение
- Верификация компании и модерация вакансий в MVP **не нужны**

Тест и грейд (критерий 25%)

- Один общий набор заданий на всех — **плохо** (утечка)
- Уникальная LLM-генерация на каждого — **риск несравнимости**
- Нужен **свой устойчивый подход** (см. docs/ARCHITECTURE.md → Assessment)
- Грейд **не режется принудительно** вниз: можно пересдать ниже/выше
- Лимит смены грейда: конфиг `GRADE_COOLDOWN_DAYS` (default 30)

Желательно, не блокер MVP

- Публикация вакансий + отклик кандидата
- Короткие регулярные задания от работодателя
- PDF резюме → разбор

За рамками кода, нужна концепция в доке

- Антифрод вакансий / недобросовестные работодатели
- ATS-интеграции
- Реальная интеграция с реестром ФСП (API не выдают; обязателен кейс «нет истории ФСП»)

Критерии жюри

Блок	Вес	Что закрывает в коде
Механика подбора	35%	need → match → invite → accept/reveal
Тест / валидация / категории	25%	survey + forms + score + category
UI	10%	два кабинета + демо-сценарий
Техника	7%	Docker, OpenAPI, безопасные секреты
Полнота + доки	6%	README, ARCHITECTURE, валидация
Доп. функционал	7%	вакансии / PDF / short tasks
Питч	10%	только финал

Демо-сценарий (обязателен)

Регистрация → опрос/тест → категория → поиск работодателя → приглашение (+ отклик, если сделан). Отдельно объяснить тест и подбор.

HandCheck — архитектура и структура репозитория

Документ для Grok Bot / coding agents. Читать вместе с docs/TZ.md и AGENTS.md.

Зафиксированные решения (не переизобретать без запроса оператора)

Тема	Решение	Почему
Форма	Модульный монолит Node 20 + Express	Уже задеплоен, быстрые итерации

Тема	Решение	Почему
UI MVP	Серверные страницы + JS в <code>app/public</code> (без SPA-фреймворка на старте)	UI весит 10%; скорость демо важнее
БД	SQLite (<code>better-sqlite3</code>) в volume <code>/data</code>	Один контейнер, простое демо
Auth	Email + magic link / одноразовый код (без Keycloak в MVP)	ТЗ: Keycloak — ориентир, не требование
Тест	Банк заданий + параллельные формы + cutoff по грейду	Устойчивее утечки и LLM-уникальности
Подбор	Сначала категория, потом <code>rank(score, fspBoost)</code>	35% жюри
Деплой	Docker Compose + CI на <code>main</code> → <code>handcheck.baski.pro</code>	Уже работает
Не делать: парсер HH/Avito, рейтинг по престижу резюме, секреты в git.		

Rev.5 (implemented)

Modular monolith with SQLite `/data`, sessions in DB, assessment battery 8+1, matching rank $0.60*test_score + 0.15*motivation + 0.10*fsp_boost + 0.15*domain_boost$, deck, calls + WS signaling, `createApp()` export.

See `docs/IMPLEMENTATION-PLAN-rev5.md`.

Целевое дерево

```
handcheck/
├── AGENTS.md           # правила для агента
├── README.md
├── package.json
├── Dockerfile
├── docker-compose.yml
├── openapi.yaml        # контракт API (добавить в фазе API)
├── docs/
│   ├── TZ.md           # сжатое ТЗ
│   ├── ARCHITECTURE.md # этот файл
│   └── DATA-MODEL.md  # сущности и поля
```

```

├── └─ VALIDATION.md          # как сами проверяем качество теста/подбора
├─ app/
│   ├── server.js            # bootstrap: middleware, mount routers, static
│   ├── config.js            # env: PORT, DB_PATH, GRADE_COOLDOWN_DAYS, ...
│   ├── db/
│   │   ├── migrate.js
│   │   ├── schema.sql
│   │   └─ seed.js           # специализации, грейды, формы теста, демо-юзеры
│   ├── middleware/
│   │   ├── auth.js
│   │   └─ errors.js
│   ├── modules/
│   │   ├── auth/            # регистрация, confirm email, session
│   │   ├── candidates/      # профиль кандидата, приватность, FSP ID stub
│   │   ├── employers/       # профиль компании
│   │   ├── catalog/         # справочник specialization × grade → category
│   │   ├── assessment/      # опрос, выдача формы, submit, score, category assign
│   │   ├── matching/        # need → candidates + explanation
│   │   ├── invitations/     # invite, statuses, accept → reveal contacts
│   │   ├── vacancies/       # OPTIONAL MVP+
│   │   └─ fsp/              # stub достижений + «нет истории ФСП»
│   ├── lib/
│   │   ├── ranking.js       # score + fspBoost
│   │   ├── privacy.js       # redaction контактов
│   │   └─ ids.js
│   └─ public/               # UI
│       ├── index.html
│       ├── candidate/
│       ├── employer/
│       └─ styles.css
│   └─ app.js                # общий fetch/helpers
└─ scripts/                  # ops (уже есть bootstrap/runner)

```

Пока модулей нет — **создавать по фазам ниже**, не одним коммитом «всё сразу».

Домены и ответственность

catalog

Справочник, который команда задаёт сама (ТЗ разрешает). MVP минимум:

- specializations: `backend`, `frontend`, `fullstack`, `qa`, `data`, `devops` (можно расширять)
- grades: `junior`, `middle`, `senior`
- `category = specialization + " × " + grade` (стабильный `category_id`)

Категория, которую видит работодатель, ставится **только** после теста, не из резюме.

assessment (25% жюри)

Поток:

1. Survey: отрасль / specialization
2. Candidate выбирает claimed grade
3. Система выдаёт **форму** из банка для (specialization, grade) — случайная из ≥ 2 параллельных форм
4. Submit $\rightarrow$ score $\rightarrow$ pass/fail cutoff
5. При pass: category_id + test_score
6. При fail: предложить форму на grade ниже (без принудительного даунгрейда текущей категории, если уже была)
7. Смена грейда: не чаще GRADE_COOLDOWN_DAYS

Форматы заданий MVP: MCQ + короткие work-sample (текст/псевдокод); автопроверка по ключу/рубрике. Judge0/запуск кода — фаза позже, не blocker первого демо.

Антиутечка: несколько форм на ячейку матрицы, не один вечный квиз.

matching (ядро 35%)

Вход: employer_need (stack[], specialization?, grade?, notes).

Выход:

```
{
  "categories": [{ "categoryId": "...", "why": "..." }],
  "candidates": [{
    "id": "...",
    "categoryId": "...",
    "rankScore": 0.87,
    "explanation": ["совпал grade middle", "test_score 0.91", "есть FSP stub"],
    "publicProfile": { "...без email/phone..." }
  }]
}
```

Правила:

- фильтр по категории теста;
- $\text{rank} = w1 * \text{test_score} + w2 * \text{fsp_boost}$ (веса в config);
- контакты только через privacy.redact до accept.

invitations

Статусы: `sent` → `viewed` → `accepted` | `declined`.

Обязательные поля: `company name`, `offer text`, `salary_from`, `salary_to` (₽).

На `accepted`: работодатель получает контакты кандидата.

fsp

Нет внешнего API. Stub: опциональный `fsp_id` + массив достижений или пусто.

UI и matching **обязаны** корректно работать при пустой истории.

vacancies (опционально)

Не блокирует MVP. Если делать — ЗП от-до обязательна; отклик раскрывает контакты как accept.

API (черновик контракта)

Префикс `/api`. JSON. После auth — session cookie или Bearer.

Method	Path	Модуль	Назначение
POST	<code>/api/auth/register</code>	auth	email + role
POST	<code>/api/auth/confirm</code>	auth	token/code
POST	<code>/api/auth/login</code>	auth	
GET	<code>/api/me</code>	auth	текущий пользователь
GET/PUT	<code>/api/candidate/profile</code>	candidates	
POST	<code>/api/assessment/survey</code>	assessment	specialization
POST	<code>/api/assessment/start</code>	assessment	claimed grade → form
POST	<code>/api/assessment/submit</code>	assessment	answers → category
GET	<code>/api/candidate/category</code>	assessment	статус + cooldown

Method	Path	Модуль	Назначение
GET	/api/candidate/invitations	invitations	
POST	/api/candidate/invitations/:id/accept	invitations	
POST	/api/candidate/invitations/:id/decline	invitations	
GET/PUT	/api/employer/profile	employers	
POST	/api/employer/needs	matching	создать/обновить потребность
GET	/api/employer/matches	matching	подборка + why
GET	/api/employer/candidates	matching	банк + фильтры
POST	/api/employer/invitations	invitations	
GET	/api/employer/invitations	invitations	
GET	/api/health	server	уже есть

Старые stub-роуты в `server.js` заменить модульными, сохранив `/api/health`.

Фазы реализации (порядок для агента)

Делать **по одной фазе**, коммит + проверка на <https://handcheck.baski.pro>.

Phase 0 — каркас (сейчас)

Шаблон UI + health. Задеплоено.

Phase 1 — данные и auth

db/schema, migrate, seed справочников, register/confirm/login, `/api/me`.

Phase 2 — assessment → category

Survey, forms bank, submit, category assign, cooldown. Кандидат видит свою категорию в UI.

Phase 3 — matching + invitations (ядро жюри)

Need, matches с explanation, invite с ЗП, accept/decline, reveal contacts. Полный демо-сценарий.

Phase 4 — polish

OpenAPI, VALIDATION.md с метриками на seed-данных, UI polish, seed демо-аккаунтов.

Phase 5 — опционально

Vacancies/applications, PDF parse stub, short tasks, code runner.

Не начинать Phase 5, пока Phase 3 не проходит демо end-to-end.

UI карта

URL	Роль	Экраны
/	public	лендинг + вход в кабинеты
/candidate/	candidate	профиль, тест, категория, приглашения
/employer/	employer	компания, потребность, подборка, банк, исходящие

Каждый экран бьёт в API выше. Не дублировать бизнес-логику в клиенте.

Конфиг (env)

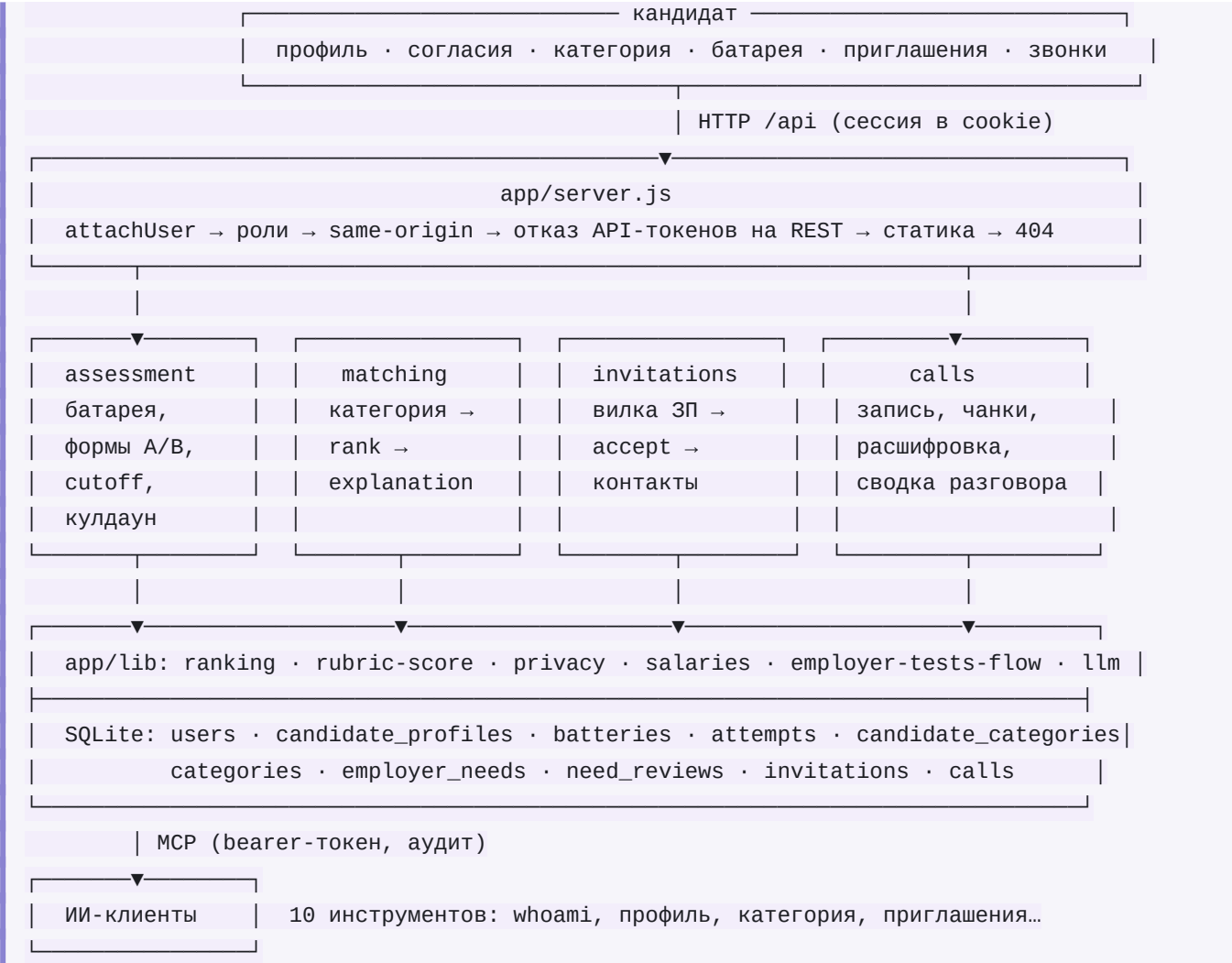
```
PORT=8810
DB_PATH=/data/handcheck.sqlite
SESSION_SECRET=...
GRADE_COOLDOWN_DAYS=30
RANK_W_TEST=0.7
RANK_W_FSP=0.3
APP_BASE_URL=https://handcheck.baski.pro
```

Секреты только в .env на сервере / CI secrets, не в репозитории.

Критерии «готово к демо»

- 1. Два аккаунта (кандидат, работодатель) из seed или регистрации.
- 2. Кандидат получает категорию только после теста.
- 3. Работодатель видит подборку с `explanation`, шлёт приглашение с вилкой.
- 4. До асепт email/phone кандидата не отдаются в API работодателю.
- 5. После асепт контакты видны.
- 6. Кейс без FSP работает.
- 7. `/api/health` зелёный на проде.

Функциональная архитектура (реализовано)



Компонентная архитектура

Компонент	Модуль	Ответственность
Auth	app/modules/auth	регистрация, код подтверждения, сессии, демо-вход
Профиль кандидата	app/modules/candidates	профиль, доступность, опыт, категория, история, приглашения
Компания	app/modules/employers	профиль компании, обзор счётчиков
Потребность	app/modules/needs	описание потребности, стек, домен, активность
Тест	app/modules/assessment	выдача батареи, шаги, дедлайны, черновики, скоринг, категория
Генерация заданий	app/modules/tasks	LLM-черновики заданий и публикация (требуется x-demo-admin в DEMO_MODE)
Подбор	app/modules/matching	пул по категории, ранжирование, фильтры, объяснения
Колода	app/modules/deck	карточка, решения «отложить/отказать», отложенные
Приглашения	app/modules/invitations	создание, статусы, раскрытие контактов, звонки
Короткие тесты работодателя	app/modules/employer-tests	создание, вопросы, публикация, назначение, ответы
Звонки	app/modules/calls	комната, запись чанками, расшифровка, анализ
Интеграции	app/modules/integrations	API-токены для MCP, аудит вызовов
MCP	app/modules/mcp	JSON-RPC сервер для ИИ-клиентов
Справочник	app/db/seed.js,	специализации, грейды, категории, 162

Компонент	Модуль	Ответственность
	app/db/battery-catalog	задания
Скоринг	app/lib/rubric-score.js, app/lib/ranking.js	рубрики, cutoff, rank
Приватность	app/lib/privacy.js, app/lib/privacy-policy.js	редакция контактов, согласия, уведомление 152-ФЗ
LLM	app/lib/llm-client.js	OpenCode Zen, space-bunny-free → glm-5.3- flash

Стек: Node.js 20+ / Express / SQLite (better-sqlite3) / статический UI без SPA-фреймворка / WebSocket для комнаты звонка. Подробности — [docs/API.md](#), [docs/DEPLOYMENT.md](#).

Механика тестирования и почему она устойчива

Документ закрывает требование ТЗ: «Единый набор заданий для всех не подходит... генерация уникальных заданий нейросетью под каждого кандидата не гарантирует сопоставимость. Придумайте и реализуйте подход, устойчивый к этим проблемам».

Выбранный подход: банк заданий + параллельные формы + cutoff по грейду

специализация × грейд → батарея (quick-вопросы + мини-проект) → рубрика → score → cutoff → категория		
9 ячеек	2 равноценные формы А и В	ключи и веса
0.55/0.68/0.78		

1. Банк заданий вместо генерации под каждого

- 9 батарей: backend | frontend | qa × junior | middle | senior.
- В каждой ячейке 18 заданий: 16 коротких (quick) + 2 проектных (work) — **162 задания**.
- У каждой задания есть рубрика: список ключевых выражений и веса, поэтому оценка воспроизводима и не зависит от того, кто и когда отвечал.
- LLM используется только для черновиков новых заданий работодателю (post

`/api/assessment/generate`), в банк они попадают после редакторской проверки. Оценка кандидата никогда не зависит от свежесгенерированного текста.

2. Параллельные формы А и В — защита от утечки ответов

- При старте батареи форма выбирается случайно (`form_key ∈ {A, B}`); обе формы собраны так, чтобы быть равноценными по сложности и рубрике.
- Проверка эквивалентности: на одном и том же наборе ответов средний балл форм А и В отличается на **0.00** при допустимом пороге 0.15 (`validate:assessment`, метрика A1).
- Проверка устойчивости к утечкам: серия стартов показывает обе формы в каждой ячейке — **9 из 9** (`scripts/assessment-metrics.js`, метрика `leakageResistance`).
- Передача ограничена: `GRADE_COOLDOWN_DAYS` (по умолчанию 30 дней) на специализацию, поэтому «выучить ответы» невыгодно.

3. Честная категоризация, а не принудительное понижение

Параметр	Значение
<code>test_score</code>	$0.6 \times \text{knowledge} + 0.4 \times \text{breadth}$, где $\text{knowledge} = 0.5 \times \text{quick} + 0.5 \times \text{work}$
Cutoff junior	0.55
Cutoff middle	0.68
Cutoff senior	0.78
Ниже cutoff	статус <code>unconfirmed</code> , категория не выдаётся, грейд не понижается
Выше cutoff	категория «специализация × грейд» фиксируется в <code>candidate_categories</code>

Измеренные значения на эталонных ответах (`context/metrics-assessment.json`):

Ответы	<code>test_score</code>	Итог против cutoff middle (0.68)
Сильные (эталон middle)	0.734	подтверждено
Слабые (эталон middle)	0.000	не подтверждено
Форма А / форма В (эталон)	0.812 / 0.812	$\Delta = 0.00$

4. Технические ограничения, которые делают замер честным

- **Серверный дедлайн.** Таймер живёт на сервере (`opened_at` + тип задачи), клиент показывает остаток, полученный от сервера. Перезагрузка страницы не продлевает время.
- **Пошаговость.** Пропустить шаг нельзя: `not_current_task`.
- **Черновик.** Ответ сохраняется до отправки (`PATCH /tasks/:id/draft`), при `pagehide` отправляется через `sendBeacon`.
- **Телеметрия попыток.** События `open/blur/focus/paste/draft/submit` пишутся в `attempt_events` и влияют на метрику `integrity`; в выдачу работодателя она не отдаётся.
- **Утечки нет.** Ответ на последнем шаге не содержит `test_score` — проверяется автотестом A4 в `validate:assessment`.

Где это в коде

Что	Файл
Выдача батареи, формы, шаги	<code>app/modules/assessment/router.js</code>
Скоринг, cutoff, категория	<code>app/modules/assessment/service.js</code> , <code>app/lib/rubric-score.js</code>
Дедлайны и целостность	<code>app/lib/attempt-timer.js</code> , <code>app/lib/attempt-integrity.js</code>
Банк заданий и сид	<code>app/db/battery-catalog/*</code> , <code>app/db/task-battery-content.js</code>
Проверки	<code>scripts/validate-assessment.js</code> , <code>scripts/assessment-metrics.js</code>

Как проверить

```
npm run validate:assessment # A1-A4: эквивалентность, cutoff, отсутствие утечек
METRICS_JSON=context/metrics-assessment.json node scripts/assessment-metrics.js # числа
для отчёта
```

Механика подбора и логика категоризации

Документ закрывает требование ТЗ: категория определяется прохождением теста, работодатель ищет по категории и выходит на конкретного кандидата, контакты скрыты до согласия.

1. Категория

```
category = специализация × грейд
backend × junior | backend × middle | backend × senior | frontend × ... | qa × ...
```

- Справочник категорий задаёт команда проекта (`categories` в SQLite, сид из `app/db/seed.js`).
- Категория присваивается **только** после батареи и только если `test_score ≥ cutoff` (см. [docs/TESTING.md](#)). Кандидат без теста виден в банке как «неподтверждённый» и всегда стоит ниже подтверждённых.
- Пересдача возможна, но не чаще раза в `GRADE_COOLDOWN_DAYS` (30 дней) по той же специализации; грейд при этом не режется принудительно.

2. Пул кандидатов для потребности

```
потребность (specialization, grade, stack, domainText)
→ кандидаты той же категории
→ rank = 0.60·test_score + 0.15·motivation + 0.10·fsp_boost + 0.15·domain_boost
→ сортировка: сначала совпадение грейда, затем rank
→ фильтры (стек, ФСП) только сужают выдачу
```

Компонент	Смысл
test_score	результат батареи, главный вклад
motivation	поведение во время мини-проекта (телеметрия, без чтения лишних данных)
fsp_boost	1, если у кандидата есть подтверждённые достижения ФСП, иначе 0
domain_boost	совпадение прошлых ролей/доменов с текстом потребности

Каждая строка подборки содержит `explanation` — текстовое объяснение, почему кандидат в этой категории и что в его ответах совпало с потребностью.

3. Колода и решения работодателя

- `GET /api/employer/needs/:id/deck/next` — одна карточка: категория, стек, опыт, объяснение, **без контактов**.
- Решения: `rejected` (отказать) и `later` (отложить). Они записываются в `need_reviews` и учитываются: приглашение отклонённому не отправить (409 `candidate_rejected`), отложенный попадает в `GET /needs/:id/deferred` и может быть возвращён в работу.
- Фильтры в подборке (`stack`, `fsp`) не меняют состав уже полученной выдачи — это проверяется аудитом (E-07).

4. Приглашение и приватность

```
работодатель → приглашение (вилка от-до, текст оффера, канал связи)
                ↓ status = sent
кандидат      → accept | decline
                ↓ accept
контакты      → видны работодателю (phone, contact_email)
```

- Вилка ЗП обязательна: без `salaryFrom/salaryTo` приглашение не создаётся (400).
- Вакансия не требуется — приглашение создаётся по потребности.
- До `accept` в JSON работодателя нет ни `phone`, ни `contact_email`, а `GET /api/employer/candidates/:id/contacts` отдаёт 403.
- Повторное приглашение по той же потребности тому же кандидату — 409 (`invitation_duplicate`).

5. Что можно расширить

Идея	Статус
Публикация вакансий и отклик кандидата	не сделано, каталог <code>app/modules/vacancies/</code> пуст
Регулярные короткие задания от работодателя	механика коротких тестов есть (<code>employer-tests</code>), периодичность — в планах
Реальный реестр ФСП	схема адаптера описана в docs/FSP-INTEGRATION.md

Идея	Статус
ATS-концепция	описана как концепция в docs/FUNCTIONAL-COVERAGE.md

Где это в коде

Что	Файл
Ранжирование	<code>app/lib/ranking.js</code> , <code>app/lib/domain-boost.js</code> , <code>app/lib/category-status.js</code>
Пул и фильтры	<code>app/modules/matching/pool.js</code> , <code>app/modules/matching/candidates-list.js</code>
Колода и решения	<code>app/modules/deck/router.js</code>
Приглашения и раскрытие контактов	<code>app/modules/invitations/*</code>
Приватность выдачи	<code>app/lib/privacy.js</code>

Как проверить

```
npm run validate:matching # M1 категория, M2 score, M3 ФСП, M4 нет контактов в matches
npm run validate:privacy # P1-P8: контакты, согласия, токены, лимиты загрузки
BASE_URL=http://127.0.0.1:8899 node scripts/functional-audit.js # механика end-to-end
```

Процедура валидации и её результаты

ТЗ требует «собственную процедуру валидации тестов и описанные результаты». Ниже — что измеряем, чем и какие числа получились на сид-данных.

1. Качество теста

Метрика	Что проверяет	Порог	Результат
A1 — эквивалентность форм A/B	одинаковые ответы дают одинаковый балл на обеих формах	$\Delta < 0.15$	0.812 / 0.812, $\Delta = 0.00$ ✓
A3 — чувствительность	эталонные «сильные» ответы проходят cutoff	≥ 0.68 (middle)	0.734 ✓
A2 — селективность	эталонные «слабые» ответы не проходят	< 0.68	0.000 ✓
A4 — нет утечки	ответ на последнем шаге не содержит балла и метрик	нет <code>test_score</code> , <code>integrity</code>	✓
Устойчивость к утечкам	в каждой ячейке реально выдаются обе формы	обе формы ≥ 2 ключа	9 из 9 ячеек за 6 выдач ✓
Состав банка	сколько заданий и как распределены формы	—	9 батарей, 162 задания (по 16 quick + 2 work, A = 9 / B = 9 в каждой ячейке)

```
npm run validate:assessment  
METRICS_JSON=context/metrics-assessment.json node scripts/assessment-metrics.js
```

2. Качество подбора

Метрика	Что проверяет	Результат
M1	подборка по потребности backend × middle содержит только Backend × Middle	✓
M2	при равной категории выше тот, у кого больше test_score	✓
M3	при равном score выше кандидат с ФСП (<code>RANK_W_FSP > 0</code>)	✓
M4	до accept в подборке нет phone / contact_email	✓

```
npm run validate:matching
```

3. Приватность

Метрика	Что проверяет	Результат
P1	работодатель не видит приватные поля кандидата	✓
P1b	контакты скрыты, обращение быстро отклоняется (403)	✓
P2–P6	согласия, ограничения по контенту, разграничение ролей	✓
P7	идентификаторы звонков — UUID	✓
P8	лимит загрузки записи (413 до буферизации)	✓

```
npm run validate:privacy
```

4. Сквозная функциональная проверка

Прогон	Что делает	Результат
<code>node scripts/functional-audit.js</code> (режим <code>full</code>)	поднимает сценарий на своём стенде с чистой БД: регистрация → согласие → батарея → категория → кулдаун → потребность → подборка → фильтры → приглашение → ассерт → контакты → decline → колода → короткий тест работодателя → MCP → LLM	41 PASS, 0 FAIL
<code>AUDIT_MODE=readonly node scripts/functional-audit.js</code>	только чтение на развёрнутом стенде	8 PASS, 0 FAIL

Отчёты: [context/audits/](#), числа метрик: `context/metrics-assessment.json`.

5. Тесты и ограничения

Команда	Результат
<code>npm test</code>	347 pass / 9 fail — те же 9 падений воспроизводятся на чистом HEAD (миграции и rubric-тесты, не связанные с этим проектом)

Команда	Результат
<code>npm run validate</code>	все метрики проходят

Ограничения, которые нужно называть честно:

- Оценка заданий — ключевая (rubric-based), а не экспертная: это воспроизводимо, но не заменяет ревью живого специалиста.
- Сид-данные синтетические, статистики по «живым» кандидатам нет.
- FSP-буст реализован как флаг и ранжирование, сетевой интеграции с реестром нет — схема описана в [docs/FSP-INTEGRATION.md](#).

Не заявляем «точность 99%»: приведённые выше числа — это пороги процедуры, а не точность модели.

Схема интеграции с реестром достижений ФСП

Публичного API реестра ФСП на момент разработки нет, поэтому интеграция спроектирована как **адаптер с тремя режимами работы**. Кейс «у кандидата нет истории ФСП» — штатный, а не ошибка: он не блокирует ни тест, ни категорию, ни приглашение.

1. Текущее состояние

Слой	Что есть сейчас
Данные	поле <code>fsp_id</code> в <code>candidate_profiles</code> , таблица <code>fsp_achievements</code> (схема подготовлена)
Ранжирование	<code>fsp_boost = 1</code> , если достижения подтверждены; иначе 0 (<code>app/lib/ranking.js</code>)
Проверка	<code>validate:matching</code> M3 — при равном score кандидат с ФСП выше
UI	фильтр «есть ФСП» в банке кандидатов (<code>?fsp=1</code>)
Импорт данных	отсутствует: сведения вносятся вручную оператором

2. Целевая схема



Нормализованная модель достижения

```
{
  "fspId": "12345",
  "source": "registry",
  "fetchedAt": "2026-10-09T10:00:00Z",
  "verified": true,
  "items": [
    { "kind": "contest", "name": "Итоги Всероссийской олимпиады", "place": 3, "year": 2025 },
    { "kind": "activity", "name": "Летняя школа", "year": 2024 }
  ]
}
```

Три режима адаптера

Режим	Когда	Поведение
live	реестр доступен и OAuth проходит	достижения подтягиваются по <code>fsp_id</code> , результат кэшируется на 24 ч
stale	кэш есть, реестр недоступен	используем последний успешный ответ, помечаем дату в интерфейсе
offline	реестр недоступен и кэша нет	<code>fsp_boost = 0</code> , кандидат работает без ФСП — без ошибок и блокировок

Переключение режима — флагом `FSP_MODE=auto|live|offline`, деградация не должна приводить к отказу в доступе: отсутствие достижений ФСП не влияет на прохождение теста и выдачу категории.

3. Ограничения и безопасность

- Токен хранится только на сервере, в UI и в ответах API не выдаётся.
- Запрос к реестру идёт с серверного таймаута и rate-limit, ответ валидируется по схеме (`zod`).
- Публичный профиль кандидата показывает только факт наличия достижений и их вид (соревнование / активность), без внутренних идентификаторов реестра.
- Согласие на обработку данных фиксируется в `privacy_consent`s до первого запроса.

4. Что нужно для включения

1. Зарегистрировать приложение в реестре ФСП и получить `clientId/clientSecret`.
2. Заполнить `FSP_CLIENT_ID`, `FSP_CLIENT_SECRET`, `FSP_API_URL`, `FSP_MODE`.
3. Реализовать `refreshFspProfile(userId)` и вызов из `PUT /api/candidate/profile`.
4. Расширить `validate:matching` сценарием «реестр недоступен → категория и подбор работают».

Сейчас этот пункт закрыт концепцией: схема, модель данных, ранжирование и режим деградации реализованы, сетевой вызов — нет, потому что публичного API не существует.

Описание API

Контракт машинно-читаемый — [openapi.yaml](#) (OpenAPI 3). Этот документ объясняет логику: кто что вызывает, что скрыто от кого и где ломается контракт.

Карта маршрутов

Все маршруты под `/api`, кроме `/mcp` (JSON-RPC для ИИ-клиентов). Аутентификация — сессия в cookie `handcheck_sid`; для MCP — bearer-токен из раздела «Интеграции».

Область	Методы и пути	Роль
Auth	<code>POST /api/auth/register</code> , <code>POST /api/auth/confirm</code> , <code>POST /api/auth/login</code> , <code>POST /api/auth/logout</code> , <code>POST /api/auth/demo-login</code>	все
Профиль кандидата	<code>GET PUT /api/candidate/profile</code> , <code>GET PUT /api/candidate/availability</code> , <code>GET POST DELETE /api/candidate/background</code>	кандидат
Категория и история	<code>GET /api/candidate/category</code> , <code>GET /api/candidate/past</code> , <code>GET /api/stats</code>	кандидат
Тест	<code>POST /api/assessment/battery/start</code> , <code>GET /api/assessment/battery/current</code> , <code>GET /api/assessment/tasks/:id</code> , <code>POST /api/assessment/tasks/:id/open</code> , <code>PATCH POST /api/assessment/tasks/:id/draft</code> , <code>POST /api/assessment/tasks/:id/submit</code> , <code>POST /api/assessment/events</code>	кандидат
Генерация заданий	<code>POST /api/assessment/generate</code> , <code>POST /api/assessment/tasks/:id/publish</code>	работодатель, в <code>DEMO_MODE</code> — заголовок <code>x-demo-admin: 1</code>
Приглашения (кандидат)	<code>GET /api/candidate/invitations</code> , <code>POST /api/candidate/invitations/:id/accept</code>	кандидат

Область	Методы и пути	Роль
Приглашения (работодатель)	/api/candidate/invitations/:id/decline	работодатель
	POST /api/employer/invitations, GET	
	/api/employer/invitations, GET	
	/api/employer/invitations/calls, GET	
	/api/employer/candidates/:candidateId/contacts	
Компания и потребность	GET PUT /api/employer/profile, GET POST	работодатель
	/api/employer/needs, PUT /api/employer/needs/:id, GET	
	/api/employer/dashboard, GET /api/employer/overview	
Подбор	GET /api/employer/needs/:id/matches, GET	работодатель
	/api/employer/candidates, GET	
	/api/employer/needs/:id/deck/next, GET	
	/api/employer/needs/:id/deferred, POST	
	/api/employer/needs/:id/reviews, DELETE /api/employer/needs/:id/reviews/:candidateId	
Короткие тесты	POST GET /api/employer/tests, GET PUT DELETE	работодатель
	/api/employer/tests/:id, POST	
	/api/employer/tests/:id/items, PUT DELETE	
	/api/employer/tests/:id/items/:itemId, POST	
	/api/employer/tests/:id/items/reorder, POST	
	/api/employer/tests/:id/publish, POST	
	/api/employer/tests/:id/generate, POST /api/employer/tests/:id/assign, GET /api/employer/test- assignments/:id, GET /api/employer/company-tests	
Короткие тесты (кандидат)	GET /api/candidate/company-tests, GET	кандидат
	/api/candidate/company-tests/:assignmentId, POST	
	/api/candidate/company-tests/:assignmentId/start, POST	
	/api/candidate/company-tests/:assignmentId/answers, POST /api/candidate/company-tests/:assignmentId/submit	
Звонки	POST /api/calls/:id/start, POST /api/calls/:id/consent,	обе роли по приглашению
	POST /api/calls/:id/transcript-chunk, POST	
	/api/calls/:id/end, GET /api/calls/:id/recording, GET	
	/api/calls/:id/analysis, GET /api/calls/for-	

Область	Методы и пути	Роль
	<code>invitation/:invitationId</code>	
Интеграции	<code>GET /api/integrations/config</code> , <code>GET POST /api/integrations/tokens</code> , <code>DELETE /api/integrations/tokens/:id</code> , <code>GET /api/integrations/audit</code>	работодатель
Приватность	<code>GET /api/privacy-notice</code>	все
MCP	<code>POST /mcp (tools/list, tools/call)</code>	bearer-токен
Служебные	<code>GET /api/health</code> , <code>GET /api/me</code>	все

Правила доступа

- 1. **Роль.** Кандидат получает 403 на `/api/employer/*`, работодатель — на кандидатские эндпоинты теста (кроме чтения профиля/категории там, где это нужно).
- 2. **Подтверждение email.** Все рабочие маршруты требуют `email_confirmed_at`.
- 3. **Same-origin.** Небезопасные методы с чужим `origin` отклоняются (403) — защита от CSRF.
- 4. **Приватность.** Контакты кандидата не отдаются работодателю, пока нет принятого приглашения.
- 5. **Баллы.** `test_score`, `integrity`, `trust_ok` не появляются ни в одном публичном ответе.
- 6. **Токены.** API-токен действует только на `/mcp`; на REST-запрос с токеном — 403.

Формат ошибок

```
{ "error": "cooldown",  
  "message": "Пересдача по этой специализации пока недоступна",  
  "retakeAt": "2026-11-08T12:15:46.142Z" }
```

Код	Когда
400 <code>invalid_body</code>	не прошла валидация; в <code>details.fields</code> — что именно исправить
401 <code>unauthorized</code>	нет сессии
403 <code>forbidden</code>	нет роли, нет согласия, контакты до аксепт, токен на REST

Код	Когда
404 <code>not_found</code>	чужой объект или несуществующий маршрут
409	конфликт состояния: <code>cooldown</code> , <code>invitation_duplicate</code> , <code>candidate_rejected</code> , <code>not_current_task</code> , <code>deadline_passed</code>
413	запись звонка больше лимита (80 МБ)
501 <code>llm_not_configured</code>	нет ключа LLM для генерации черновиков
502 <code>llm_failed</code>	LLM недоступен или вернул невалидный JSON

Пример сквозного вызова

```
# 1. кандидат начал батарею
curl -c c.txt -X POST localhost:8810/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"anna@demo.local","password":"demo-demo-demo"}'
curl -b c.txt -X POST localhost:8810/api/assessment/battery/start \
  -H 'Content-Type: application/json' \
  -d '{"specialization":"backend","grade":"middle","privacyConsent":true}'

# 2. работодатель смотрит подборку (контактов в ней нет)
curl -c e.txt -X POST localhost:8810/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"cafe@demo.local","password":"demo-demo-demo"}'
curl -b e.txt localhost:8810/api/employer/needs/<needId>/matches

# 3. приглашение с вилкой → контакты появятся только после accept
curl -b e.txt -X POST localhost:8810/api/employer/invitations \
  -H 'Content-Type: application/json' \
  -d '{"needId":"<needId>","candidateId":"<candidateId>","salaryFrom":120000,
    "salaryTo":180000,"offerText":"Приглашаем в команду","contactChannel":"telegram"}'
```

HandCheck — модель данных (MVP)

SQLite. Имена таблиц — snake_case. UUID или prefix_timestamp id допустимы; предпочтительно UUID v4.

users

Поле	Тип	Notes
id	TEXT PK	
email	TEXT UNIQUE	
email_confirmed_at	TEXT NULL	ISO
role	TEXT	candidate employer
password_hash	TEXT NULL	или magic-link only
created_at	TEXT	

sessions / auth_tokens

Одноразовые коды подтверждения email и сессии. Не логировать токены.

candidate_profiles

Поле	Тип	Notes
user_id	TEXT PK FK	
display_name	TEXT	
headline	TEXT	
stack_json	TEXT	JSON array
phone	TEXT NULL	sensitive
contact_email	TEXT NULL	sensitive ; default = users.email
privacy_json	TEXT	что видно до invite
fsp_id	TEXT NULL	stub

Поле	Тип	Notes
consent_at	TEXT	NULL

employer_profiles

Поле	Тип	Notes
user_id	TEXT	PK FK
company_name	TEXT	
description	TEXT	
industry	TEXT	
contact_email	TEXT	

specializations / grades

Справочники seed. `categories` = декартово или явная таблица `category_id`, `specialization`, `grade`, `label`.

assessment_forms

Поле	Тип	Notes
id	TEXT	PK
specialization	TEXT	
grade	TEXT	
form_key	TEXT	А/В/... параллельные формы
items_json	TEXT	задания + ключи/рубрика
pass_score	REAL	cutoff 0..1

Поле	Тип	Notes
------	-----	-------

Уникальность: (specialization, grade, form_key).

assessment_attempts

Поле	Тип	Notes
------	-----	-------

id	TEXT PK	
candidate_user_id	TEXT	
form_id	TEXT	
claimed_grade	TEXT	
specialization	TEXT	
answers_json	TEXT	
score	REAL NULL	
passed	INTEGER NULL 0/1	
started_at / submitted_at	TEXT	

candidate_categories

Текущая видимая категория (после успешного теста).

Поле	Тип	Notes
------	-----	-------

candidate_user_id	TEXT PK	
category_id	TEXT	
specialization	TEXT	

Поле	Тип	Notes
grade	TEXT	
test_score	REAL	для ранжирования
assigned_at	TEXT	
grade_changed_at	TEXT	для cooldown

fsp_achievements

Поле	Тип	Notes
id	TEXT PK	
candidate_user_id	TEXT	
title	TEXT	
kind	TEXT	contest/event/...
year	INTEGER NULL	
evidence_note	TEXT	stub

Пустой набор = валидный кейс «нет истории ФСП».

employer_needs

Поле	Тип	Notes
id	TEXT PK	
employer_user_id	TEXT	
title	TEXT	
specialization	TEXT NULL	

Поле	Тип	Notes
grade	TEXT NULL	
stack_json	TEXT	
notes	TEXT	
active	INTEGER	
created_at	TEXT	

invitations

Поле	Тип	Notes
id	TEXT PK	
employer_user_id	TEXT	
candidate_user_id	TEXT	
need_id	TEXT NULL	
company_name	TEXT	snapshot
offer_text	TEXT	
salary_from	INTEGER	P, required
salary_to	INTEGER	P, required
status	TEXT	sent/viewed/accepted/declined
created_at / updated_at	TEXT	

Инвариант: salary_from <= salary_to, оба > 0.

vacancies / applications (Phase 5)

Только если Phase 3 готов. У vacancy тоже salary_from/salary_to.

Правила видимости (код + SQL)

Функция publicCandidateView(viewer, candidate):

- viewer = сам кандидат → всё своё
- viewer = employer и **нет** accepted invite / application → без phone/contact_email; можно display_name по privacy, category, stack, test_score band, fsp flag
- viewer = employer и есть accepted → контакты открыты

API работодателя **никогда** не должен отдавать сырой JOIN на contacts без этой проверки.

Сборка, развёртывание и локальный запуск

Требование ТЗ: «Пошаговая инструкция сборки, развёртывания и локального запуска».

1. Требования к машине

Параметр	Значение
CPU	4–16 vCPU
RAM	2–8 ГБ (рекомендуется 32–64 ГБ, если жюри гоняет нагрузочные сценарии)
GPU	не требуется
Диск	5 ГБ + том для SQLite
ПО	Docker 24+ и Docker Compose v2 (или Node.js 20+ для локального запуска)

ffmpeg нужен только для склейки записей звонков; без него остальная система работает.

2. Локальный запуск из исходников

```
git clone git@github.com:handcheck:BaskovKonstantin/handcheck.git
cd handcheck
cp .env.example .env
npm install
npm start # http://127.0.0.1:8810
```

Ключевые переменные `.env`:

Переменная	Назначение	По умолчанию
PORT	порт приложения	8810
DB_PATH	путь к SQLite	/data/handcheck.sqlite
SESSION_SECRET	подпись сессий	обязательна в проде
GRADE_COOLDOWN_DAYS	лимит пересдачи	30
APP_BASE_URL	внешний адрес для ссылок	http://127.0.0.1:8810
DEMO_MODE	демо-аккаунты и код подтверждения 000000	1
LLM_BASE_URL, LLM_API_KEY	генерация черновиков заданий	Zen (space-bunny-free → glm-5.3-flash)

Секреты в репозиторий не попадают: `.env` в `.gitignore`, наружу отдаётся только `.env.example`.

3. Docker Compose (рекомендуемый способ)

```
docker compose up -d --build
curl -fsS http://127.0.0.1:8810/api/health
```

- Секреты берутся из `.env` через `env_file`, БД живёт в томе `handcheck-data:/data`.
- Контейнер публикует только `127.0.0.1:8810`; внешний адрес закрывает Caddy.

4. Развёртывание

```
push в main → self-hosted runner (метка handcheck) → docker compose up -d --build
→ health-опрос до 120 с с проверкой GIT_COMMIT
```

Состав деплоя — [.github/workflows/deploy.yml](#). Проверка после раскатки:

```
curl -fsS http://127.0.0.1:8810/api/health # с самого сервера (hairpin по домену
может не работать)
BASE_URL=https://handcheck.baski.pro AUDIT_MODE=readonly node scripts/functional-audit.js
```

5. Демо-данные и сценарий для жюри

```
DEMO_MODE=1 DB_PATH=/data/handcheck.sqlite node scripts/seed-jury-pack.js
```

Email	Роль	Состояние
anna@demo.local	кандидат	подтверждённая категория Backend × Middle
boris@demo.local	кандидат	та же категория, часто «отложен» в колоде
demo3@demo.local ... demo8@demo.local	кандидаты	разный test_score, разный порядок в колоде
demo-unconf@demo.local	кандидат	тест не проходил — видно, как категория не выдаётся
cafe@demo.local	работодатель	потребность «Автоматизация работы официанта»

Пароль — из `.env.example`, код подтверждения — `000000`. Клик-маршрут: [docs/JURY-DEMO.md](#).

6. Библиотеки и версии

Пакет	Версия	Зачем
express	^4.21.2	HTTP-сервер и маршрутизация
better-sqlite3	^11.8.1	синхронный SQLite в одном процессе
bcryptjs	^2.4.3	хеширование паролей
zod	^3.25.76	валидация тела запросов и MCP-инструментов

Пакет	Версия	Зачем
<code>ws</code>	<code>^8.18.1</code>	сигналинг комнаты звонка
<code>multer</code>	<code>^2.0.2</code>	приём записей звонков (чанки)
<code>@modelcontextprotocol/sdk</code>	<code>^1.32.1</code>	MCP-сервер для ИИ-клиентов
<code>@fontsource-variable/manrope, onest</code>	<code>^5.3.0/^5.3.1</code>	шрифты интерфейса без внешних CDN
<code>playwright</code>	<code>^1.49.1 (dev)</code>	функциональные и UI-проверки
<code>supertest</code>	<code>^7.1.0 (dev)</code>	HTTP-тесты API

Системные: Node.js ≥ 20, ffmpeg (записи звонков), Docker Compose (деплой).

7. Проверки перед сдачей

```
npm test # 347 проходят; 9 известных падений есть и на чистом HEAD
npm run validate # качество теста, подбора и приватности
node scripts/functional-audit.js # 41 проверка на стенде
BASE_URL=https://handcheck.baski.pro AUDIT_MODE=readonly node scripts/functional-audit.js
# 8 проверок на проде
METRICS_JSON=context/metrics-assessment.json node scripts/assessment-metrics.js
# числа для отчёта
```

HandCheck MCP

HandCheck exposes a [Model Context Protocol](#) server at `/mcp` (Streamable HTTP). Personal API tokens authenticate requests; each token inherits the owner’s role (candidate or employer) and chosen scopes.

Quick start

1. Open **Интеграции** in your cabinet (`/candidate/integrations` or `/employer/integrations`).
2. Create a token with **Чтение** and, if needed, **Запись**. Copy the `hc_...` value — it is shown once.

- Point your AI client at `https://handcheck.baski.pro/mcp` (or your local `http://127.0.0.1:8810/mcp`) with header `Authorization: Bearer hc_....`

Cursor (`.cursor/mcp.json`)

```
{
  "mcpServers": {
    "handcheck": {
      "url": "https://handcheck.baski.pro/mcp",
      "headers": {
        "Authorization": "Bearer hc_YOUR_TOKEN"
      }
    }
  }
}
```

Claude Code

```
claude mcp add --transport http handcheck https://handcheck.baski.pro/mcp \
  --header "Authorization: Bearer hc_YOUR_TOKEN"
```

Clients without HTTP MCP

```
export HANDCHECK_URL=https://handcheck.baski.pro
export HANDCHECK_TOKEN=hc_YOUR_TOKEN
node bin/handcheck-mcp.js
```

Register the command as a stdio MCP server in Claude Desktop or other tools.

Scopes

Scope	Meaning
read	List/read tools, resources, prompts; no mutations
write	Includes read; allows profile updates, assessment submit, deck decisions, invitations

Tools by role

Common: `whoami`

Candidate: `get_my_profile`, `update_my_profile`, `get_my_category`, `list_tasks`, `get_task`, `start_assessment`, `submit_answer`, `submit_work_task`, `list_invitations`, `respond_invitation`,

`list_calls`

Employer: `list_needs`, `create_need`, `update_need`, `get_next_candidate`, `decide_candidate`,
`list_shortlist`, `list_invitations`, `list_calls`, `get_call_analysis`

Resources & prompts

- `handcheck://profile`
- `handcheck://needs/{id}` (employer)
- `handcheck://tasks/{id}` (candidate, attempt id)
- Prompts: `candidate-next-steps`, `employer-hiring-flow`

Privacy & product rules

- No numeric test scores or integrity in tool results intended for candidates.
- Contacts stay hidden until invitation accept (same as the web app).
- Actions via MCP are stored with `source: mcp`; employers see «через ИИ-клиент» on invitations created by MCP.
- MCP calls are rate-limited and logged in **Журнал MCP** on the Integrations page.

Audit

Each tool invocation writes one row: tool name, success/failure, short summary. Revoked tokens stop working immediately.

Jury demo — HandCheck (FSP / ЛЦТ)

Короткий сценарий для оценки MVP без доступа к продакшен-данным.

Окружение для оценки

- **RAM:** ~32–64 GB
- **CPU:** 4–16 vCPU
- **GPU:** не требуется (0 GPUs) — весь стек Node.js + SQLite + ffmpeg для записей звонков
- Локально: `npm start` или `docker compose up` (см. комментарии в `docker-compose.yml`)

Демо-аккаунты (DEMO_MODE=1)

Email	Роль	Заметка
anna@demo.local	Кандидат	Подтверждённая категория Backend × Middle, сильный domain boost под кафе
boris@demo.local	Кандидат	Та же категория, в колоде кафе часто «отложен»
demo3@demo.local ... demo8@demo.local	Кандидаты	Разный test_score в сиде — порядок в колоде
demo-unconf@demo.local	Кандидат	Неподтверждён — тест не проходил, виден в поиске ниже подтверждённых
cafe@demo.local	Работодатель	Потребность «Автоматизация официанта»

Пароль: из .env.example (demo-demo-demo). Код email: 000000.

Click-path (~10 минут)

1. **Работодатель** — вход cafe@demo.local → /employer/deck.
2. Листайте колоду: сверху кандидаты с подтверждённой категорией и выше rank (Анна с domain boost обычно выше demo3–8).
3. Внизу — **«Backend × Middle — неподтверждён»** (demo-unconf@demo.local): профиль не скрыт, но ниже в ранжировании.
4. Пилюля «неподтверждён» — пунктирная, приглушённая.
5. **Список совпадений** — /employer/list для той же потребности: те же метки categoryStatus / подпись категории.
6. **Кандидат без категории** — demo-unconf@demo.local → /candidate/today: статус неподтверждён, СТА «Пройти тест».
7. **Кандидат с категорией** — anna@demo.local → «Сегодня»: подтверждённая категория, приглашения.
8. **Регистрация и 152-ФЗ** — /auth?mode=register: дата рождения, согласие (чекбокс не предзаполнен), для 15–17 — согласие представителя; /privacy с оператором по умолчанию.

9. **Тест** — `anna` или новый кандидат → `/candidate/tasks`: короткие вопросы **60 с по серверному дедлайну** (после перезагрузки таймер = остаток с сервера). Cooldown пересдачи **30 дней** (`GRADE_COOLDOWN_DAYS`).

Что проверить в ранжировании

- Подтверждённые (`categoryStatus: confirmed`) всегда выше неподтверждённых при прочих равных.
- Среди подтверждённых: выше `test_score` + `domain_boost` (опыт в домене потребности) + FSP.
- Провал теста **не снимает** уже подтверждённую категорию (повторная батарея → cooldown, старая категория в API кандидата).
- Контакты в колоде скрыты до ассепт приглашения (`validate-privacy`).

API (для жюри-технарей)

- `GET /api/candidate/category` → `status: confirmed | unconfirmed`
- `GET /api/employer/needs/{id}/deck/next` → `card.categoryStatus, card.categoryLabel`
- Список матчей — те же поля в элементах.